



INSTITUTE OF AERONAUTICAL ENGINEERING

(Autonomous)

Dundigal - 500 043, Hyderabad, Telangana

COURSE CONTENT

FRONT-END WEB DEVELOPMENT LABORATORY								
I Semester: CSE / IT / CSE (AI&ML) / CSE (DS)								
Course Code	Category	Hours / Week			Credits	Maximum Marks		
ACSE04	Foundation	L	T	P	C	CIA	SEE	Total
		0	0	2	1	40	60	100
Contact Classes: Nil	Tutorial Classes: NIL	Practical Classes: 30			Total Classes: 30			
Prerequisite: There are no prerequisites to take this course.								

I. COURSE OVERVIEW:

The Front-End Web Development Laboratory course provides hands-on experience in building responsive and interactive web applications using HTML5, CSS3, and JavaScript. Students will learn to design structured web pages, apply styling and layout techniques, and implement client-side validations. The course introduces modern frameworks like Bootstrap/Tailwind CSS for responsive design and covers DOM manipulation, event handling, and API integration. Through practical experiments and mini-projects, learners develop problem-solving skills and creativity in web design. By the end of the lab, students will be able to design, validate, and deploy user-friendly front-end interfaces aligned with industry standards.

II. COURSES OBJECTIVES:

The students will try to learn

- I Apply HTML5 and CSS3 to design structured, styled, and responsive web pages.
- II Implement client-side validation, event handling, and DOM manipulation using JavaScript.
- III Analyze CSS frameworks such as Bootstrap and Tailwind to develop responsive layouts.
- IV Integrate APIs and asynchronous JavaScript techniques to create dynamic web applications.
- V Design a mini-project demonstrating creativity and usability in front-end development.

III. COURSE OUTCOMES:

At the end of the course students should be able to:

- CO 1 Identify the basic structure and components of a web page using HTML5 elements.
- CO 2 Apply CSS3 properties and layouts to enhance the presentation of web pages.
- CO 3 Implement JavaScript for client-side validation, event handling, and interactivity.
- CO 4 Analyze different CSS frameworks to choose appropriate tools for responsive web design.
- CO 5 Develop APIs and asynchronous JavaScript features to build dynamic content-driven applications.
- CO 6 Implement APIs and asynchronous JavaScript features to build dynamic content-driven applications

IV. COURE CONTENT:

The course is organized into 14 modules, each corresponding to one week of study. The structure ensures a gradual progression from fundamental concepts to advanced topics, with hands-on exercises and project work integrated throughout.

1. Module 1: Introduction & Web Basics

- (a) Overview of how the web works and the client-server model.
- (b) Frontend vs Backend vs Full Stack development.
- (c) Setting up the development environment (VS Code, Git, browser tools).
- (d) Creating and testing the first HTML page.

2. Module 2: HTML Fundamentals

- (a) HTML elements and attributes.
- (b) Semantic tags and document structure.
- (c) Forms and input elements.
- (d) Tables and multimedia.
- (e) Accessibility features (alt text, ARIA basics).

3. Module 3: CSS Fundamentals

- (a) CSS syntax and inclusion methods.
- (b) Selectors and combinators.
- (c) Box model, margins, padding, and borders.
- (d) Typography and colors.
- (e) Introduction to responsive design.

4. Module 4: CSS Layouts

- (a) Positioning and display types.
- (b) Floats and clear.
- (c) Flexbox (deep dive).
- (d) Grid layout (deep dive).
- (e) Media queries for responsiveness.

5. Module 5: Advanced CSS & UI Design

- (a) Transitions, transforms, and animations.
- (b) Pseudo-classes and pseudo-elements.
- (c) CSS variables and custom properties.
- (d) Introduction to CSS frameworks (Bootstrap, TailwindCSS).

6. Module 6: JavaScript Basics

- (a) Variables, data types, and operators.
- (b) Functions and scope.
- (c) DOM manipulation.
- (d) Events and event handling.
- (e) Debugging with browser console.

7. Module 7: JavaScript Control Flow

- (a) Arrays and objects.
- (b) Loops and conditionals.
- (c) Event listeners and form validation.
- (d) Mini-projects (e.g., calculator, form validator).

8. Module 8: JavaScript ES6+ Features

- (a) let and const.
- (b) Arrow functions.
- (c) Template literals and destructuring.
- (d) Spread and rest operators.
- (e) Modules and imports.

9. Module 9: Asynchronous JavaScript

- (a) Callbacks and promises.
- (b) async/await.
- (c) Fetch API for AJAX requests.
- (d) Working with JSON.
- (e) Mini-project: Weather App or API-based app.

10. Module 10: Advanced JavaScript Topics

- (a) Closures and higher-order functions.
- (b) DOM traversal.
- (c) LocalStorage and SessionStorage.
- (d) Error handling in JavaScript.
- (e) Introduction to bundlers (Vite, Webpack).

11. Module 11: Introduction to React

- (a) Why frameworks? SPA vs MPA.
- (b) React basics: components and JSX.
- (c) Props and state.
- (d) Using React Developer Tools.

12. Module 12: React Advanced Concepts

- (a) React Hooks: useState and useEffect.
- (b) Handling forms in React.
- (c) React Router for navigation.
- (d) API calls within React.

13. Module 13: Styling in React & Project Work

- (a) Styling with CSS Modules.
- (b) Styled Components and TailwindCSS.
- (c) Component libraries (Material UI, shadcn/ui).

14. Module 14: Capstone Project & Deployment

- (a) Integrating all concepts into a final project.
- (b) Using Git and GitHub for collaboration.
- (c) Deployment on Netlify, Vercel, or GitHub Pages.
- (d) Final project presentation and review.

V. PLATFORMS FOR WEB DEVELOPMENT

1. GitHub Pages

Website: <https://pages.github.com/>

GitHub Pages is a free hosting service for static websites directly from a GitHub repository.

It supports HTML, CSS, and JavaScript, making it ideal for deploying simple front-end projects. Developers can automate deployments using Git workflows and integrate CI/CD pipelines. It provides custom domain support and HTTPS security by default. However, it is not suitable for dynamic applications that require server-side processing.

2. Netlify

Website: <https://www.netlify.com/>

Netlify is a powerful platform for deploying static websites and JAMstack applications. It offers features like continuous deployment from Git, built-in form handling, and serverless functions. Netlify's free plan provides generous limits, making it suitable for small to medium-sized projects. It also includes automatic HTTPS, global CDN, and rollbacks for deployments. The platform supports integrations with popular tools like Next.js, Gatsby, and React.

3. Vercel

Website: <https://vercel.com/>

Vercel is a cloud platform optimized for front-end frameworks like Next.js, React, and Vue.js. It simplifies deployment with automatic builds and serverless functions. With a global edge network, Vercel ensures fast performance and scalability. It supports custom domains, environment variables, and preview deployments for collaboration. The platform is widely used by developers building modern web applications.

4. Firebase Hosting

Website: <https://firebase.google.com/>

Firebase Hosting is a Google-powered service designed for fast and secure static and dynamic web app deployment. It offers integration with Firebase's real-time database, authentication, and cloud functions. The free tier includes a generous amount of bandwidth and SSL encryption. It provides a seamless developer experience with one-command deployment. Firebase Hosting is ideal for Progressive Web Apps (PWAs) and mobile-friendly websites.

5. Heroku

Website: <https://www.heroku.com/>

Heroku is a cloud platform-as-a-service (PaaS) that enables developers to deploy full-stack applications effortlessly. It supports multiple programming languages, including Node.js, Python, and Ruby. Heroku provides add-ons for databases, caching, and monitoring, enhancing application scalability. Its free tier includes dynos (containers) that run apps with occasional sleeping periods. While easy to set up, Heroku is best suited for prototyping and small projects due to resource limitations.

6. CodeSandbox

Website: <https://codesandbox.io/>

CodeSandbox is an online IDE that provides an interactive environment for front-end and full-stack web development. It supports frameworks like React, Vue, and Angular, allowing developers to quickly prototype and share their projects. With real-time collaboration and live preview, CodeSandbox is ideal for testing and showcasing web applications. Its integration with GitHub enables seamless version control and deployment.

7. Glitch

Website: <https://glitch.com/>

Glitch is a collaborative coding platform that allows developers to create, edit, and deploy web applications instantly. It provides a real-time editor and automatic hosting for Node.js applications, making it an excellent

choice for beginners. Glitch supports full-stack development with built-in databases and API integrations. Its remixable projects allow users to build upon existing applications effortlessly.

8. Replit

Website: <https://replit.com/>

Replit is an online IDE that supports multiple programming languages and frameworks. It offers cloud-based execution, enabling users to write, test, and deploy applications from any device. Replit provides real-time collaboration, making it ideal for pair programming and classroom learning. With built-in hosting and database support, Replit simplifies full-stack development.

9. WordPress

Website: <https://wordpress.com/>

WordPress is a widely used CMS that enables users to build and customize websites with ease. It offers responsive themes and plugins, making it a versatile platform for blogging, business sites, and e-commerce. The free plan allows users to create and host a basic website with WordPress branding. For more control, developers can self-host WordPress on their own server.

10. Wix

Website: <https://www.wix.com/>

Wix is a user-friendly website builder that provides drag-and-drop functionality for creating responsive websites. It offers a wide range of templates and customization options, making it ideal for beginners. Wix provides free hosting and domain services, allowing users to publish their websites instantly. While it is best for static sites, advanced users can add custom functionality using Wix Velo, a built-in development platform.

VI. SOFTWARE REQUIREMENTS

To implement web development mini-projects using the listed platforms, certain software requirements must be met. These requirements vary based on the specific tools and technologies used.

Basic Requirements

1. Web Browser: Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari (for testing and development).
2. Internet Connection: Required for cloud-based platforms like GitHub Pages, Netlify, Vercel, Firebase, etc.

Development Tools

1. Code Editor: Visual Studio Code, Sublime Text, or Atom (for writing HTML, CSS, JavaScript).
2. Terminal/Command Line: For running Git commands, package managers (npm, yarn), or deploying applications.
3. Git: For version control and hosting on platforms like GitHub and GitHub Pages.

Additional Software Based on Project Needs

1. Node.js & npm: Required for JavaScript-based frameworks (React, Vue, Angular, Next.js).
2. Python & Pip: Needed for Flask/Django-based web applications.
3. PHP & MySQL: If working with WordPress self-hosted sites or traditional backend development.
4. Docker: If containerizing applications for platforms like Heroku or Firebase.

VII. HARDWARE REQUIREMENTS

The hardware requirements depend on the complexity of the web development projects and the tools being used. Below are the recommended specifications:

Basic Requirements

1. Processor: Intel Core i3 (or equivalent) or higher.
2. RAM: Minimum 4GB (8GB or more recommended for smooth development).
3. Storage: At least 20GB of free space (SSD recommended for faster performance).
4. Display: Minimum resolution of 1366×768 (Full HD recommended).
5. Internet Connection: Required for cloud-based platforms and online development tools.

Recommended for Advanced Development

1. Processor: Intel Core i5/i7 or AMD Ryzen 5/7 for faster build times.
2. RAM: 16GB or more for handling multiple applications (e.g., Docker, Node.js, databases).
3. Storage: SSD with at least 256GB+ for better performance.
4. Graphics Card: Not necessary for basic web development, but useful for UI/UX design and graphics-intensive applications.

Additional Peripherals (Optional)

1. External Monitor: For multitasking and better UI/UX design experience.
2. Mechanical Keyboard & Mouse: For improved efficiency in coding.
3. Webcam & Microphone: If collaborating on projects via video conferencing.

EXERCISES: 1 - INTRODUCTION & WEB BASICS

Learning Outcomes

By completing these exercises, students will:

1. Configure a working development environment.
2. Understand and use browser DevTools effectively.
3. Create and publish their first web page.
4. Apply Git and GitHub for version control and project hosting.

1 Environment Setup

1. Install VS Code and add extensions such as Live Server and Prettier.
2. Install Git and configure your username and email.
3. Create a GitHub account.
4. Clone a sample repository from GitHub and open it in VS Code.

2 First HTML Page

1. Create a file named index.html.
2. Add the following elements:
 - (a) A page title using the <title> tag.
 - (b) A heading with your name.
 - (c) A paragraph introducing yourself.
 - (d) An external link to your favorite website.
3. Open the page in a browser.

3 Browser DevTools Exploration

1. Open your index.html file in Chrome or Firefox.
2. Right-click and select Inspect to open DevTools.
3. Modify the heading text directly in the Elements panel.
4. Change the paragraph color using the Styles panel.
5. Switch to the Network tab and reload the page to observe requests.

4 Git Basics

1. Initialize a Git repository in your project folder using git init.
2. Stage and commit your index.html file with:

```
git add.  
git commit -m "Created first HTML page"
```
3. Create a repository on GitHub named frontend-lab1.
4. Push your local repository to GitHub.

5 Mini Project – Course Introduction Page

1. Create a web page that includes:
 - (a) A heading: “Frontend Development Course”.
 - (b) A paragraph describing the course in your own words.
 - (c) An unordered list () of at least three things you expect to learn.
 - (d) A footer with your name and the date.
2. Upload this project to GitHub.
3. (Optional) Enable GitHub Pages and share the hosted link.

EXERCISES: 2 - HTML FUNDAMENTALS

Learning Outcome

By completing these exercises, students will:

1. Write valid and structured HTML documents.
2. Use semantic tags for better readability and accessibility.
3. Design forms with various input types and labels.
4. Create and format tables with headers and merged cells.
5. Integrate multimedia elements into web pages.
6. Develop a portfolio skeleton page as a foundation for future projects.

1 HTML Structure Practice

1. Create a new HTML5 file with the proper `<!DOCTYPE html>` declaration.
2. Add the basic structure including `<html>`, `<head>`, and `<body>`.
3. Inside the body, include:
 - (a) A main heading (`<h1>`) and a subheading (`<h2>`).
 - (b) A paragraph describing yourself.
 - (c) An ordered list of three favorite hobbies.
 - (d) An unordered list of three favorite foods.
 - (e) A hyperlink to your favorite website.

2 Semantic HTML Page

1. Create a blog-style page using semantic tags.
2. Include:
 - (a) A `<header>` with the blog title.
 - (b) A `<nav>` with three links (Home, About, Contact).
 - (c) A `<main>` containing one `<article>` with a heading and two paragraphs.
 - (d) An `<aside>` with a short author bio.
 - (e) A `<footer>` with your name and year.

3 Forms in HTML

1. Design a registration form with the following fields:
 - (a) Name (text input).
 - (b) Email (email input).
 - (c) Password (password input).
 - (d) Gender (radio buttons).
 - (e) Hobbies (checkboxes).
 - (f) Country (dropdown menu).
2. Add a Submit and Reset button.
3. Ensure each input has a corresponding label.

4 Tables

1. Create a student marks table with the following columns:
 - (a) Name
 - (b) Subject
 - (c) Marks
 - (d) Grade
2. Include table headers using `<th>`.
3. Use `colspan` and `rowspan` in at least one place.
4. Add borders and cell padding for readability.

5 Multimedia Integration

1. Insert an image with appropriate alt text.
2. Embed a YouTube video using `<iframe>`.
3. Add an audio clip with playback controls.

6 Mini Project - Portfolio Skeleton

1. Create the skeleton of a personal portfolio page using only HTML.
2. The page should include:
 - (a) An `<header>` with your name and a navigation menu.

- (b) An <section> for skills (list of at least 5 skills).
- (c) An <section> for education (table format with institution, degree, year).
- (d) A <section> with a simple contact form (Name, Email, Message).
- (e) A <footer> with your name and copyright.

EXERCISES: 3 - CSS FUNDAMENTALS

Learning Outcome

By completing these exercises, students will:

1. Apply CSS to style HTML elements using inline, internal, and external methods.
2. Use various CSS selectors to target elements effectively.
3. Understand and manipulate the CSS box model.
4. Style text, lists, and links for better readability and design.
5. Create visually appealing personal pages using CSS.

1 Applying Basic CSS Styles

1. Create an HTML page with headings, paragraphs, and lists.
2. Apply CSS styles using:
 - (a) Inline CSS for one element.
 - (b) Internal CSS using <style> tag.
 - (c) External CSS by linking a separate CSS file.
3. Change the following styles:
 - (a) Text color, background color, and font size.
 - (b) Font family and font style (italic, bold).
 - (c) Text alignment (left, center, right).

2 CSS Selectors

1. Use different types of CSS selectors:
 - (a) Element selector
 - (b) Class selector
 - (c) ID selector
 - (d) Group selector
 - (e) Descendant and child selectors
2. Apply distinct styles to demonstrate each selector.

3 Box Model Practice

1. Create a few <div> boxes with content.
2. Apply the following CSS properties:
 - (a) Width and height
 - (b) Padding, margin, and border
 - (c) Background color and border style
3. Observe how the box model affects layout and spacing.

4 Typography and Colors

1. Experiment with font families, sizes, weights, and styles.
2. Apply different text colors and background colors.
3. Use text decoration (underline, overline, line-through).
4. Experiment with letter-spacing and line-height.

5 Lists and Links Styling

1. Create ordered and unordered lists.
2. Change bullet styles for unordered lists and numbering styles for ordered lists.
3. Style links with different pseudo-classes:
 - (a) :link
 - (b) :visited
 - (c) :hover
 - (d) :active

6 Mini Project – Styled Personal Page

1. Take the HTML page created in Module 2 mini project (Portfolio Skeleton).
2. Apply CSS to improve styling:
 - (a) Set colors, fonts, and text alignment.
 - (b) Apply padding, margin, and borders for sections.
 - (c) Style the navigation menu and links with hover effects.
 - (d) Add background colors or images to sections.

EXERCISES: 4 - CSS LAYOUTS

Learning Outcome

By completing these exercises, students will:

1. Understand and apply different CSS positioning methods.
2. Use display and float properties effectively.
3. Build layouts using Flexbox and Grid techniques.
4. Make web pages responsive for multiple screen sizes using media queries.
5. Create complex, structured, and adaptive web page layouts.

1 Positioning Practice

1. Create an HTML page with multiple <div> elements.
2. Apply different CSS positioning properties:
 - (a) static (default)
 - (b) relative
 - (c) absolute
 - (d) fixed
 - (e) sticky
3. Observe how each positioning type affects layout.

2 Display and Float

1. Experiment with display properties:
 - (a) block
 - (b) inline
 - (c) inline-block
 - (d) none
2. Use float to align elements to the left or right.
3. Clear floats using the clear property.

3 Flexbox Layout

1. Create a container with multiple child boxes.
2. Apply Flexbox properties on the container:
 - (a) display: flex;
 - (b) flex-direction (row, column, row-reverse, column-reverse)
 - (c) justify-content (flex-start, center, space-between, space-around)
 - (d) align-items (flex-start, center, stretch)
 - (e) flex-wrap (wrap, nowrap)
3. Style individual flex items using order, flex-grow, and flex-shrink.

4 CSS Grid Layout

1. Create a container with multiple boxes.
2. Apply Grid layout properties:
 - (a) display: grid;
 - (b) Define columns and rows using grid-template-columns and grid-template-rows.
 - (c) Set gaps using grid-gap.
 - (d) Position items using grid-column and grid-row.
 - (e) Experiment with auto-fit and auto-fill for responsive grids.

5 Responsive Design with Media Queries

1. Create a page with a header, sidebar, content area, and footer.
2. Use media queries to modify layout for different screen sizes:

- (a) Make sidebar collapse below content on small screens.
- (b) Change font size and padding for mobile devices.
- (c) Adjust grid or flex layouts for tablet and mobile views.

6 Mini Project - Responsive Layout Page

1. Design a webpage (e.g., a simple blog or portfolio page) with:
 - (a) Header with navigation menu
 - (b) Sidebar for links or information
 - (c) Main content area
 - (d) Footer
2. Apply Flexbox or Grid for the layout.
3. Make the page responsive using media queries.

EXERCISES: 5 - ADVANCED CSS & UI DESIGN

Learning Outcome

By completing these exercises, students will:

1. Apply transitions, transforms, and animations to enhance user interfaces.
2. Use pseudo-classes and pseudo-elements for styling.
3. Implement CSS variables for maintainable and consistent styles.
4. Utilize CSS frameworks to quickly build responsive and styled components.
5. Create visually appealing and interactive web pages.

1 CSS Transitions

1. Create a page with multiple buttons or boxes.
2. Apply a transition effect on hover:
 - (a) Change background color smoothly.
 - (b) Change text color smoothly.
 - (c) Apply multiple property transitions (e.g., color and border).
3. Experiment with transition-duration and transition-timing-function.

2 CSS Transforms

1. Use transform properties on images or boxes:
 - (a) `scale()` to resize elements.
 - (b) `rotate()` to rotate elements.
 - (c) `translate()` to move elements.
 - (d) `skew()` to tilt elements.
2. Combine multiple transforms for creative effects.

3 CSS Animations

1. Create keyframe animations for a box or image:
 - (a) Animate position (left, top).
 - (b) Animate background color or text color.
 - (c) Animate scale or rotation.
2. Control animation using animation-duration, animation-delay, and animation-iteration-count.

4 Pseudo-classes and Pseudo-elements

1. Style links with pseudo-classes:
 - (a) `:hover`
 - (b) `:focus`
 - (c) `:active`
2. Use pseudo-elements to enhance design:
 - (a) `::before` to add content before elements.
 - (b) `::after` to add content after elements.
3. Create decorative effects like underlines, icons, or background shapes.

5 CSS Variables and Custom Properties

1. Define CSS variables for colors, fonts, and spacing.
2. Use variables throughout the stylesheet for consistency.
3. Change variable values and observe effects on the page.

6 Introduction to CSS Frameworks

1. Use Bootstrap or TailwindCSS to style a simple page:
 - (a) Create a navigation bar.
 - (b) Create a responsive card layout with images and text.
 - (c) Add buttons with framework classes.
 - (d) Apply spacing and color utilities from the framework.

7 Mini Project - Styled Responsive Page

1. Take your previous portfolio or layout page.
2. Apply advanced CSS techniques:
 - (a) Add transitions and hover effects to buttons and links.
 - (b) Use transforms and animations for interactive sections.
 - (c) Apply pseudo-elements for decorative effects.
 - (d) Use CSS variables for consistent styling.
 - (e) Optionally, implement Bootstrap/Tailwind components.

EXERCISES: 6 - JAVASCRIPT BASICS

Learning Outcome

By completing these exercises, students will:

1. Understand basic JavaScript syntax, variables, and data types.
2. Perform operations using operators and expressions.
3. Write functions with parameters and return values.
4. Manipulate the DOM to change content and styles dynamically.
5. Implement event-driven interactions to make web pages interactive.
6. Identify and fix JavaScript errors using the browser console.
7. Understand the difference between log, warn, error, and debugger.
8. Gain confidence in inspecting variables, functions, and DOM elements in real-time.
9. Develop practical interactive projects such as a To-Do List and a Calculator.

1 Task : JavaScript Setup and Basic Output

1. Create a new HTML file and link an external JavaScript file.
2. Display a message using:
 - (a) alert()
 - (b) console.log()
 - (c) Writing to the document using document.write()
3. Experiment with comments and whitespace in JavaScript.

2 Variables and Data Types

1. Declare variables using var, let, and const.
2. Work with different data types: string, number, boolean, null, undefined.
3. Practice type conversion between strings and numbers.
4. Print the variables and their types in the console.

3 Operators and Expressions

1. Perform arithmetic operations (+, -, *, /,
2. Use comparison operators (==, ===, !=, !==, <, >, <=, >=).
3. Practice logical operators (&&, ||, !).
4. Display results using console.log().

4 Functions and Scope

1. Create functions to perform simple calculations (e.g., sum of two numbers).
2. Pass parameters to functions and return values.
3. Explore function scope and local vs global variables.
4. Call functions from HTML buttons using onclick.

5 DOM Manipulation

1. Access HTML elements using:
 - (a) document.getElementById()

- (b) `document.getElementsByClassName()`
- (c) `document.querySelector()`
- 2. Change element content using `innerHTML`.
- 3. Change element styles dynamically.
- 4. Add or remove elements from the DOM.

6 Event Handling

- 1. Create HTML buttons or input fields.
- 2. Add event listeners using `onclick`, `onmouseover`, `onchange`.
- 3. Change content or style in response to events.
- 4. Create a simple interactive feature, e.g., button changes text color on click.

7 Debugging with Browser Console

- 1. Open your HTML page with linked JavaScript in the browser.
- 2. Write a small JavaScript program with intentional errors, for example:
 - (a) Using an undefined variable.
 - (b) Performing invalid arithmetic (e.g., divide by zero).
 - (c) Misspelling a function name.
- 3. Use `console.log()` to display variable values at different stages.
- 4. Use `console.warn()` and `console.error()` to categorize messages.
- 5. Insert the debugger statement in a function and observe code execution step by step in the console.
- 6. Use `console.table()` to display an array of objects (e.g., students with name and marks).
- 7. Inspect and fix the errors by reading the console messages.

8 Mini Projects

Project 1: Interactive To-Do List

- (a) Create an input field for new tasks and a “Add Task” button.
- (b) Display the task list dynamically using an unordered list (`<ul>`).
- (c) Add a “Delete” button next to each task to remove it.
- (d) Change the style of completed tasks when clicked (e.g., `strikethrough`).
- (e) Use functions, event listeners, and DOM manipulation for all actions.

Project 2: Simple Calculator

- (a) Create a basic calculator layout with input fields for numbers and buttons for operations (+, -, *, /).
- (b) Display the result dynamically on the page.
- (c) Use JavaScript functions to perform calculations.
- (d) Handle invalid input (e.g., division by zero) gracefully.
- (e) Enhance interactivity using event listeners and dynamic updates.

EXERCISES: 7 - JAVASCRIPT CONTROL FLOW

Learning Outcome

By completing these exercises, students will:

- 1. Manipulate arrays and objects effectively in JavaScript.
- 2. Implement loops and conditional statements for various tasks.
- 3. Use event listeners to create interactive web pages.
- 4. Validate forms dynamically and provide user feedback.
- 5. Develop practical mini-projects including a Calculator, Form Validator, and Student Grades Dashboard.

1 Working with Arrays

- 1. Create an array of numbers and:
 - (a) Find the sum and average of all elements.
 - (b) Find the largest and smallest numbers.
 - (c) Sort the array in ascending and descending order.
 - (d) Reverse the array.

2. Create an array of strings (names of students) and:

- (a) Add and remove elements at the beginning and end.
- (b) Search for a particular name using a loop.
- (c) Join all names into a single string using `join()`.

2 Working with Objects

1. Create an object representing a student with properties: Name, Age, Marks (array).
2. Access and modify object properties using dot and bracket notation.
3. Loop through the object properties using for...in and display the values.
4. Create an array of student objects and:
 - (a) Calculate each student's total and average marks.
 - (b) Find the student with the highest total marks.
 - (c) Display all student details dynamically in a table.

3 Loops and Conditionals

1. Use for, while, and do-while loops to:
 - (a) Print numbers 1–50, even numbers, and odd numbers.
 - (b) Generate a multiplication table for a given number.

2. Use if-else to:

- (a) Check whether a number is positive, negative, or zero.
- (b) Determine eligibility for voting based on age input.
- (c) Check if a number is prime.

3. Use switch to:

- (a) Display day of the week based on a number (1–7).
- (b) Display month name based on a number (1–12).

4 Event Listeners

1. Add a click event to a button to display an alert message.
2. Add a mouseover and mouseout event to change the color of a paragraph.
3. Create multiple buttons dynamically and attach a click event to each using event delegation.
4. Use a keyup event to dynamically display the number of characters typed in an input field.

5 Form Validation

1. Create a registration form with Name, Email, Password, Age fields.
2. Validate:
 - (a) Name is not empty.
 - (b) Email contains @ and a valid domain.
 - (c) Password length is at least 6 characters.
 - (d) Age is between 18 and 60.
3. Display success or error messages dynamically without refreshing the page.
4. Use loops and conditionals to check multiple inputs at once.

6 Mini-Projects

Project 1: Calculator

- (a) Create a simple calculator with two input fields and buttons for +, -, *, /.
- (b) Use functions, conditionals, and event listeners to perform operations.
- (c) Display results dynamically on the page.
- (d) Add validation for empty inputs or invalid operations.

Project 2: Form Validator

- (a) Build a registration form with multiple input fields.
- (b) Validate inputs using JavaScript event listeners and conditionals.
- (c) Display inline error messages dynamically.
- (d) Prevent form submission if validation fails using event.preventDefault().

Project 3: Student Grades Dashboard

- (a) Create an array of student objects with Name, Marks.
- (b) Calculate total and average marks using loops.
- (c) Display results in a table with conditional formatting (e.g., highlight students with marks ≥ 80).
- (d) Allow adding a new student dynamically using a form and update the table.

EXERCISES: 8 - JAVASCRIPT ES6+ FEATURES

Learning Outcome

By completing these exercises, students will:

1. Differentiate between var, let, and const.
2. Write cleaner code using arrow functions.
3. Use template literals and destructuring for concise syntax.
4. Apply spread and rest operators in arrays, objects, and functions.
5. Organize code into modules and import/export them.
6. Build practical mini-projects using modern ES6+ features.

1 let and const

1. Declare variables using var, let, and const, and test scope differences inside a block.
2. Try reassigning a const variable and observe the error message.
3. Create a constant object and:
 - (a) Add new properties.
 - (b) Modify existing properties.
 - (c) Attempt to reassign the whole object and record the result.
4. Use let in a loop to correctly capture the loop index in a closure and compare it with var.

2 Arrow Functions

1. Convert a traditional function to an arrow function:
 - (a) Function to square a number.
 - (b) Function to calculate factorial using a loop.
 - (c) Function to return the maximum of two numbers.
2. Use arrow functions with map(), filter(), and reduce():
 - (a) Double each element in an array.
 - (b) Filter even numbers from an array.
 - (c) Calculate the sum of all elements in an array.
3. Demonstrate how arrow functions handle this differently than regular functions inside an object.

3 Template Literals and Destructuring

1. Use template literals to:
 - (a) Print a multi-line string.
 - (b) Embed expressions like addition or string concatenation inside a string.
 - (c) Create a string that displays user details dynamically (name, age, city).
2. Apply array destructuring:
 - (a) Extract first, second, and remaining elements from an array.
 - (b) Swap two numbers without using a temporary variable.
3. Apply object destructuring:
 - (a) Extract properties from a student object.
 - (b) Use default values when a property is missing.
 - (c) Destructure nested objects (e.g., address inside student).

4 Spread and Rest Operators

1. Use the spread operator to:
 - (a) Clone an array.
 - (b) Merge two arrays into a new one.
 - (c) Spread a string into an array of characters.
2. Use the spread operator with objects:
 - (a) Merge two objects.
 - (b) Override properties in the target object.
3. Use the rest operator to:
 - (a) Write a function that accepts variable number of arguments and returns their sum.
 - (b) Collect remaining array elements during destructuring.

5 Modules and Imports

1. Create a module file `mathUtils.js` that exports:
 - (a) A constant `PI`.
 - (b) Functions for addition, subtraction, multiplication, division.
2. Import the module in `app.js` using:
 - (a) Named imports.
 - (b) Import all as a single object.
 - (c) Default export for a special function.
3. Create another module `studentUtils.js` that exports:
 - (a) A function to calculate average marks.
 - (b) A function to find the top student from an array.
4. Import both `mathUtils.js` and `studentUtils.js` into `main.js` and demonstrate usage.

6 Mini-Projects

Project 1: Expense Tracker (using ES6 features)

- (a) Maintain a list of expenses as objects with name, amount, and date.
- (b) Use array methods and arrow functions to calculate total expenses.
- (c) Display expense details using template literals.
- (d) Use spread operator to add new expenses dynamically.

Project 2: Weather App Data Parser

- (a) Create a sample weather data object with nested details.
- (b) Use destructuring to extract temperature, humidity, and location.
- (c) Use template literals to generate a formatted weather report.

Project 3: Modular To-Do List

- (a) Create a `todo.js` module that exports functions to add, remove, and list tasks.
- (b) In `main.js`, import these functions and build a simple text-based To-Do app.
- (c) Use arrow functions, rest parameters, and template literals where appropriate.

EXERCISES: 9 - ASYNCHRONOUS JAVASCRIPT

Learning Outcome

At the end of this lab session, students will be able to:

1. Implement callback functions to manage asynchronous execution.
2. Develop programs using Promises to handle success and error states.
3. Apply `async/await` to simplify asynchronous workflows.
4. Use the Fetch API to retrieve data from external services.
5. Parse and manipulate JSON data in JavaScript applications.
6. Design and build a mini project (e.g., Weather App or API-based application) that integrates multiple asynchronous concepts.

1 Understanding Callbacks

1. Write a function that accepts a callback and prints a message before and after executing it.
2. Simulate fetching user data using `setTimeout()` and handle the result with a callback.

2 Promises in Action

1. Create a Promise that resolves after 2 seconds with a success message.
2. Create a Promise that rejects with an error and handle it using `.catch()`.
3. Chain multiple `.then()` calls to transform the result step by step.

3 Using `async/await`

1. Rewrite the Promise from Exercise 2 using `async/await`.
2. Create an async function that waits for multiple Promises using `Promise.all()`.
3. Introduce error handling using `try...catch` with async functions.

4 Fetch API Basics

1. Use the Fetch API to request data from a public API (e.g., <https://jsonplaceholder.typicode>).
2. Parse the response as JSON and log it to the console.
3. Handle network errors gracefully.

5 Working with JSON

1. Convert a JavaScript object into a JSON string using `JSON.stringify()`.
2. Parse a JSON string back into an object using `JSON.parse()`.
3. Fetch a JSON file and dynamically display its data in an HTML table.

6 Mini Projects

Project 1: Weather App

- (a) Build a simple weather application using the OpenWeatherMap API (or any free weather API).
- (b) Allow the user to enter a city name in an input field.
- (c) Fetch current weather data using the Fetch API.
- (d) Display temperature, humidity, and weather description dynamically on the page.
- (e) Handle errors for invalid city names or failed API requests.

Project 2: Random Joke Generator - API-based App (Alternative)

- (a) Create a "Random Joke Generator" app using a public jokes API (e.g., <https://official-joke-api.appspot.com>).
- (b) Add a button that, when clicked, fetches a new joke using `async/await`.
- (c) Display the joke (setup and punchline) dynamically in the DOM.
- (d) Style the app for a clean UI and handle network errors.

EXERCISES: 10 - JAVASCRIPT DOM MANIPULATION

Learning Outcome

By completing these exercises, students will:

1. Select and manipulate DOM elements using JavaScript.
2. Attach and handle different types of events.
3. Dynamically create, append, and remove elements in the DOM.
4. Modify element styles and manage CSS classes.
5. Traverse and explore DOM hierarchies.
6. Build interactive mini-projects that use DOM manipulation.

1 Selecting and Modifying Elements

1. Create an HTML page with a heading, a paragraph, and a button.
2. Use `document.getElementById`, `document.querySelector`, and `document.querySelectorAll` to select elements and log them to the console.
3. Change the text of the heading and the color of the paragraph using JavaScript.
4. Modify the button label dynamically.

2 Event Handling

1. Attach a click event listener to the button that changes the background color of the page.
2. Add a double-click event to toggle between two colors.
3. Create a text input field and use the input event to display the typed text live in a paragraph below it.
4. Add a mouseover event on the heading to increase its font size temporarily.

3 Creating and Appending Elements

1. Write a script to dynamically create a list of fruits (ul with li items) and add it to the page.
2. Add a button that appends a new fruit item when clicked.
3. Create a simple form (name and email) and, upon submission, dynamically add the entered details to a list.

4 Modifying Styles and Classes

1. Create a paragraph and two buttons: "Highlight" and "Remove Highlight".
2. Use `classList.add`, `classList.remove`, and `classList.toggle` to apply/remove a CSS highlight class.
3. Change multiple CSS properties dynamically using `element.style`.
4. Experiment with hiding and showing elements using `display` property.

5 DOM Traversal

1. Create a nested HTML structure with a parent div and child elements.

2. Use `parentNode`, `children`, `firstElementChild`, and `lastElementChild` to explore the structure.
3. Traverse sibling elements using `nextElementSibling` and `previousElementSibling`.

6 Mini-Projects

Project 1: To-Do List App

- (a) Create an input box and “Add Task” button.
- (b) On button click, add the entered task as a new list item.
- (c) Add “Delete” buttons for each task.
- (d) Use CSS classes to mark tasks as completed on click.

Project 2: Image Gallery

- (a) Display a row of thumbnail images.
- (b) On clicking a thumbnail, display the full image in a larger view section.
- (c) Add “Next” and “Previous” buttons to navigate images.

Project 3: Interactive Counter

- (a) Add “+” and “-” buttons and a display showing the counter value.
- (b) Increment or decrement the value when the buttons are clicked.
- (c) Add a “Reset” button to set the counter back to 0.

EXERCISES: 11 - INTRODUCTION TO REACT

Learning Outcome

By completing these exercises, students will:

1. Understand the difference between SPA and MPA, and the role of frameworks.
2. Set up and run a React development environment.
3. Create and render components using JSX.
4. Pass data between components using props.
5. Manage component-level state using `useState`.
6. Use React Developer Tools for debugging and inspection.
7. Build small interactive apps with React.

1 Understanding Frameworks and SPA vs MPA

1. Research and write a short note comparing Single Page Applications (SPA) and Multi Page Applications (MPA).
2. Identify three popular frameworks/libraries used for building SPAs and explain why they are preferred.
3. Create a small HTML file that simulates an MPA (two separate pages linked together).
4. Convert it into a basic SPA simulation by showing and hiding sections with JavaScript.

2 React Setup and Hello World

1. Set up a new React project using Vite or Create React App.
2. Create a simple React component called `HelloWorld` that displays a greeting.
3. Render this component inside the `App` component.
4. Experiment by modifying the greeting text and observing live reload.

3 Components and JSX

1. Create two new components: `Header` and `Footer`.
2. Use JSX to include HTML-like syntax within your components.
3. Render `Header`, a content section, and `Footer` inside the `App` component.
4. Add inline styles using JSX style objects and test different formatting.

4 Props in React

1. Create a component `UserCard` that accepts `name`, `age`, and `email` as props.
2. Render multiple `UserCard` components in `App` with different user details.
3. Experiment with passing strings, numbers, and even JSX as props.
4. Add default props to handle missing values.

5 State in React

1. Create a component `Counter` with a button to increment a number using `useState`.
2. Add another button to decrement the number.
3. Extend the component to reset the counter to 0.

4. Experiment with updating state based on the previous state value.

6 React Developer Tools

1. Install React Developer Tools in your browser.
2. Open the DevTools and inspect your React application.
3. Explore the component tree and observe how props and state change in real time.
4. Debug a simple bug (e.g., missing prop or incorrect state update) using DevTools.

7 Mini-Projects

Project 1: Profile Card App

- (a) Build a ProfileCard component with props for name, role, and image.
- (b) Render multiple profile cards in a grid layout.

Project 2: Simple Counter App

- (a) Build a standalone counter app using useState.
- (b) Include increment, decrement, and reset functionality.
- (c) Style the counter using JSX and CSS.

Project 3: Greeting App

- (a) Create a form with an input field for the user's name.
- (b) Display a personalized greeting message using React state.
- (c) Add validation for empty input.

EXERCISES: 12 - REACT ADVANCED CONCEPTS

Learning Outcome

By completing these exercises, students will:

1. Understand and apply React Hooks like useState and useEffect.
2. Handle and validate forms in React using controlled components.
3. Implement navigation and routing in a React application using react-router-dom.
4. Perform API calls within React, handle asynchronous data, and manage loading/error states.
5. Build advanced React applications that combine hooks, routing, and API integration.

1 Exploring useState Hook

1. Create a component CounterApp that uses useState to track a counter value.
2. Add buttons for increment, decrement, and reset functionality.
3. Modify the app to track multiple counters (e.g., Counter A and Counter B).
4. Experiment with updating state using both direct values and callback functions.

2 Exploring useEffect Hook

1. Create a component that displays the current time and updates every second using useEffect.
2. Build a simple app where a message is logged in the console whenever a button is clicked (demonstrating dependency arrays).
3. Implement a cleanup function inside useEffect to stop an interval when a component unmounts.
4. Observe how dependency arrays affect the re-rendering of components.

3 Handling Forms in React

1. Create a form with input fields: name, email, and password.
2. Use controlled components to manage input values with useState.
3. Display the entered data dynamically below the form.
4. Add simple validation (e.g., required fields, email format check).

4 Navigation with React Router

1. Install and configure react-router-dom.
2. Create three components: Home, About, and Contact.
3. Set up routes for each component and add navigation links.
4. Implement a "Not Found" page for invalid routes.
5. Add route parameters (e.g., /user/:id) and display user details dynamically.

5 API Calls in React

1. Create a component that fetches and displays a list of posts from the JSONPlaceholder API (<https://jsonplaceholder.typicode.com/posts>).
2. Use `useEffect` to make the API call when the component mounts.
3. Display the data in a formatted list or card layout.
4. Add a loading indicator and error handling.
5. Extend the component to fetch a single post based on an ID entered by the user.

6 Mini-Projects

1. Project 1: To-Do List with Hooks
 - (a) Build a to-do list using `useState` to store tasks.
 - (b) Add features: add, delete, and mark tasks as completed.
 - (c) Persist tasks in `LocalStorage`.
2. Project 2: Multi-Page Portfolio with React Router
 - (a) Create a portfolio app with Home, Projects, and Contact pages.
 - (b) Use React Router for navigation.
 - (c) Add route parameters for displaying project details.
3. Project 3: API-Based News App
 - (a) Fetch news articles from a public API.
 - (b) Display articles in cards with title, description, and link.
 - (c) Add navigation between categories (e.g., Technology, Sports, Business) using React Router.
 - (d) Show loading and error messages appropriately.

EXERCISES: 13 - STYLING IN REACT & PROJECT WORK

Learning Outcome

By completing these exercises, students will:

1. Apply different styling techniques in React using CSS Modules, styled-components, and TailwindCSS.
2. Integrate component libraries such as Material UI and shadcn/ui into React projects.
3. Build responsive, user-friendly interfaces with modern styling approaches.
4. Develop polished mini-projects (To-Do App, Portfolio, Theme Switcher) showcasing advanced styling skills.

1 Styling with CSS Modules

1. Create a simple React component (ProfileCard) and style it using a CSS Module.
2. Apply styles for headings, paragraphs, and a profile image.
3. Demonstrate scoping by creating another component that uses a different CSS Module with the same class names.
4. Modify styles dynamically based on props (e.g., highlight a card if it's marked as "featured").

2 Styled Components

1. Install and configure styled-components.
2. Create a button component with different styles for primary and secondary variants.
3. Apply conditional styling using props (e.g., disabled state).
4. Build a reusable card component styled entirely with styled-components.

3 TailwindCSS Basics

1. Set up TailwindCSS in a React project.
2. Build a responsive navigation bar using Tailwind utility classes.
3. Style a login form with Tailwind, ensuring proper spacing, colors, and hover effects.
4. Experiment with responsive design by adjusting layouts for mobile, tablet, and desktop.

4 Using Component Libraries

1. Install Material UI and build a form with pre-designed components (e.g., TextField, Button, Card).
2. Use icons from Material UI in the application.
3. Install and configure shadcn/ui in a React project.
4. Build a dashboard layout using cards, buttons, and navigation components from shadcn/ui.

5 Mini-Projects

Project 1: Styled To-Do App

- (a) Build a To-Do list with add, delete, and mark-as-complete functionality.
- (b) Apply styling using TailwindCSS or styled-components.
- (c) Add filters for “All”, “Completed”, and “Pending” tasks.
- (d) Store tasks in LocalStorage for persistence.

Project 2: Portfolio Website

- (a) Create a personal portfolio site with sections: Home, About, Projects, and Contact.
- (b) Use Material UI or shadcn/ui for layout and components.
- (c) Style each section with either CSS Modules or TailwindCSS.
- (d) Add responsiveness to ensure mobile and desktop compatibility.

Project 3: Theme Switcher App

- (a) Build a React app with a light/dark theme toggle.
- (b) Use styled-components or TailwindCSS for theming.
- (c) Save the user’s theme preference in LocalStorage.
- (d) Apply the theme globally across multiple components.

EXERCISES: 14 - CAPSTONE PROJECT & DEPLOYMENT

Learning Outcome

By completing these exercises, students will:

- 1. Integrate all JavaScript and React concepts into a full-fledged project.
- 2. Collaborate effectively using Git and GitHub.
- 3. Gain hands-on experience in deploying React applications on modern platforms.
- 4. Present and document their project in a professional manner.

1 Project Planning and Setup

- 1. Brainstorm and finalize an idea for the capstone project (e.g., E-commerce site, Blogging platform, Social media dashboard, etc.).
- 2. Create a project structure in React using Vite or Create React App.
- 3. Define the core features, assign roles if working in teams, and document the plan in a README file.

2 Version Control with Git and GitHub

- 1. Initialize a Git repository for the project.
- 2. Create a remote repository on GitHub and push the initial code.
- 3. Practice branching by creating feature branches for different modules.
- 4. Perform pull requests and merge changes into the main branch.
- 5. Add a .gitignore file to exclude unnecessary files.

3 Building the Capstone Project

- 1. Integrate key React concepts: components, props, state, hooks, and routing.
- 2. Connect to an external API or a backend service for data (optional).
- 3. Implement form handling, error handling, and validation.
- 4. Style the project using TailwindCSS, styled-components, or Material UI.
- 5. Add authentication or local storage features if applicable.

4 Deployment

- 1. Prepare the project for production build.
- 2. Deploy the project on one platform:
 - (a) Netlify – connect GitHub repo and auto-deploy on push.
 - (b) Vercel – use CLI or GitHub integration for deployment.
 - (c) GitHub Pages – configure and deploy static site build.
- 3. Test the deployed site on different devices/browsers.
- 4. Document deployment steps in the project README.

5 Final Project Presentation

- 1. Prepare a short presentation covering:
 - (a) Project idea and objectives.
 - (b) Key features implemented.
 - (c) Technical stack used.

- (d) Challenges faced and solutions.
- 2. Showcase a live demo of the deployed project.
- 3. Submit the project repository link, deployment link, and presentation slides.

6 Capstone Project Ideas

1. Student Management System Build a full-stack web application to manage student data.

- (a) User authentication (admin, teacher, student).
- (b) CRUD operations for student records (add, edit, delete, view).
- (c) Search and filter functionality.
- (d) Report generation in PDF/Excel format.

2. E-Commerce Website Create a modern shopping website with secure checkout.

- (a) Product listing with categories and filters.
- (b) Shopping cart and checkout system.
- (c) User authentication and order history.
- (d) Payment gateway integration (dummy or sandbox).

3. Blog Platform A multi-user blogging system with interactive features.

- (a) User registration and login.
- (b) Create, update, delete, and read posts.
- (c) Add categories, tags, and featured images.
- (d) Commenting and like/dislike system.

4. Portfolio Website with CMS A portfolio site that allows non-technical users to update content.

- (a) Admin panel to manage projects and experience.
- (b) Contact form with email integration.
- (c) Option to upload images and media.
- (d) Responsive design with dark/light mode.

5. Chat Application Build a real-time chat platform for communication.

- (a) User registration and login.
- (b) One-to-one and group chat features.
- (c) Real-time message updates using WebSockets or Firebase.
- (d) Option to send emojis and files.

6. Online Quiz Platform A system where teachers can create quizzes and students can attempt them.

- (a) Question bank with multiple question types.
- (b) Timed quiz attempts with auto-submit.
- (c) Result and score calculation with analytics.
- (d) Leaderboard and ranking system.

7. Task Management Tool Develop a Kanban-style productivity app.

- (a) Create, assign, and track tasks.
- (b) Drag-and-drop Kanban board.
- (c) Deadlines with notifications.
- (d) Collaboration features with multiple users.

8. Learning Management System (LMS) A complete platform for delivering online courses.

- (a) Course and lesson creation.
- (b) Video lectures and file uploads.
- (c) Student progress tracking.
- (d) Quizzes and certificate generation.

9. Event Booking Application A system to browse and book events online.

- (a) Event listing with search and filters.
- (b) User registration for booking tickets.
- (c) Ticket generation (QR code or unique ID).
- (d) Calendar integration for upcoming events.

10. Health & Fitness Tracker Track and visualize fitness activities.

- (a) User registration and personalized dashboard.
- (b) Daily workout and diet logging.
- (c) Graphs and charts for progress visualization.
- (d) Goal setting and achievement badges.

V. TEXT BOOKS:

- 1. Duckett, Jon. "HTML & CSS: Design and Build Websites", John Wiley & Sons, 1st Edition, 2011.
- 2. Duckett, Jon. "JavaScript and JQuery: Interactive Front-End Web Development", John Wiley & Sons, 1st Edition, 2014.
- 3. Wieruch, Robin. "The Road to Learn React: Your journey to master plain yet pragmatic React.js", Independently Published, 1st Edition, 2017

VI. REFERENCE BOOKS:

- 1. Robbins, Jennifer Niederst. "Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics", O'Reilly Media, 5th Edition, 2018.
- 2. Crockford, Douglas. "JavaScript: The Good Parts", O'Reilly Media, 1st Edition, 2008.
- 3. Thomas, Mark Tielens. "React in Action", Manning Publications, 1st Edition, 2018.

VII. ELECTRONICS RESOURCES:

- 1. https://developer.mozilla.org/en-US/docs/Learn_web_development
- 2. <https://www.freecodecamp.org>
- 3. <https://www.codecademy.com>
- 4. <https://www.frontendmentor.io>
- 5. <https://www.w3schools.com>

VIII. MATERIALS ONLINE;

- 1. Course Content
- 2. Lab manual